



Groth16: The gold standard of SNARKs

Lecture: ?

SIL763-7163

Introduction To Blockchains, Cryptocurrencies, And Smart Contracts



1. Why Verify? The Quantitative Reality

- The Law of Blockchain: "Don't Trust, Verify."
- The Scalability Bottleneck:
 - Native Execution: $x^2 + 5$ on a CPU costs $< \$0.00001$.
 - On-Chain (Ethereum): Same logic can cost \$50–\$200 in Gas fees.
- The Goal: A ~200 Byte certificate that proves a 1-hour calculation is correct.
- Efficiency: Verification time is **Constant** (does not grow with program size).



2. The Groth16 Master Pipeline

1. **Program Logic** (The "High-Level" Code) → **START**
2. **Arithmetic Circuit** (Addition/Multiplication Gates)
3. **R1CS** (Rank-1 Constraint System - The Matrix)
4. **QAP** (Quadratic Arithmetic Program - Polynomials)
5. **Cryptographic Proof** (The Bilinear Pairings)



3. Step 1: Program to Gates (Concept)

- **Arithmetization:** Flattening code into "Arithmetic Gates."
- **Allowed Gates:** Addition (+) and Multiplication ($\times$).
- **Boolean vs. Arithmetic:** CPU logic (AND/OR) is replaced by algebra over a large Prime Field.
- **The Constraint:** Multiplication is "Expensive" (it increases polynomial degree). Addition is "Free."



4. The Canonical Example ($x^2 + 5 = y$)

The Goal: Prove "I know a secret x such that the result is y ."

The Flattened Gates:

1. **Gate 1 (Multiplication):** $x \times x = v_1$
2. **Gate 2 (Addition):** $v_1 + 5 = y$

The Variables:

- x : **Private Input** (The secret "Witness").
- y : **Public Output** (The result both parties see).
- v_1 : **Internal Wire** (Intermediate work).



5. Scaling: From 2 Gates to Zcash

- Our Example: 2 Gates.
- Zcash Sapling: ~200,000 Multiplication Gates.
- The Proof Size: Both proofs are **exactly the same size** (~200 bytes).
- Link: [Zcash Sapling Design Docs](#)



6. The "Liar" Problem: Why We Need a Setup

- **Verification Paradox:** Checking every gate manually is $O(n)$. We lose efficiency.
- **Spoofing:** Without a "Scaffold," a Prover could send random points that satisfy one gate but aren't part of a valid circuit.
- **The CRS:** We need a **Common Reference String**. This is a "Public Mathematical Menu" that forces the Prover to use the circuit's specific logic.



7. The Secret Solution: Schwartz-Zippel

- **The Goal:** Prove two polynomials are the same without checking every point.
- **Intuition:** Two different parabolas can only touch at most in 2 places.
- **The Fingerprint:** If we check the polynomials at **one random secret point** (τ), and they match, they are almost certainly the same everywhere.
- **Probability:** In a field of size 2^{256} , the chance of "accidental matching" at τ is lower than the chance of a meteor hitting the classroom.



8. The "Toxic Waste" Ceremony (CRS)

- **The Ingredients:** $\tau, \alpha, \beta, \gamma, \delta$.
 - τ (**Tau**): The evaluation point (The secret spot).
 - $\alpha, \beta, \gamma, \delta$: Knowledge & Consistency keys (The "DNA" of the circuit).
- **The Danger:** If the Prover knows $\tau, \alpha, \beta, \gamma$, or δ , they can forge proofs.
- **Multi-Party Computation (MPC):** 100 people contribute "shards" of randomness.
- **Trust:** If **one** person destroys their shard, the secrets are unrecoverable.



9. Step 2: Gates to R1CS (The Matrix)

- Equation: $Lw \cdot Rw = Ow$
- L, R, O: Matrices defining the circuit "logic."
- **Canonical Example (Gate 1: $x \cdot x = v_1$):**
 - The matrices have a "1" in the column for x (Left and Right) and a "1" in the column for v_1 (Output).
- **The Wall:** We still have m rows for m gates. Still $O(n)$ to check!



10. The Witness (w)

- **Definition:** The full "Execution Trace" of the circuit.
- **Canonical Example:** $w = [1, y, x, v_1]$
 - e.g., $w = [1, 14, 3, 9]$
- **Role:** w contains every number that was on every wire of the circuit.
- **Precision:** The Prover must possess a valid w where $Lw \cdot Rw = Ow$ for all gates.



11. Compression via QAP & Lagrange

- **The Goal:** One polynomial identity to rule them all.
- **Why 3 Equations?** We need to represent **Left**, **Right**, and **Output** across the whole circuit.
- **Lagrange Interpolation:** We draw a curve ($A(x)$) that hits the values of the "Left" inputs at every gate point ($x = 1, 2, \dots, m$).
- **Identity:** If $A(x) \cdot B(x) - C(x) = H(x) \cdot Z(x)$, the circuit is valid.



12. Precision: Roots of Unity

- **Interpolation Points:** We don't use $x = 1, 2, 3$. We use **Roots of Unity** (ω^i).
- **Why?** Allows the **Fast Fourier Transform (FFT)**.
- **Efficiency:** Moves Prover time from $O(n^2)$ to $O(n \log n)$.
- **In Practice:** Without this, the Prover would take hours to generate a proof for a simple transfer.



13. The Bilinear Pairing (e)

- **The Tool:** A "Mathematical Bridge" between two elliptic curve groups (G_1 and G_2).
- **The Property:** $e(G^a, H^b) = e(G, H)^{ab}$.
- **Role in Groth16:** It allows the Verifier to multiply two "hidden" numbers (a and b) exactly once.
- **Recap:** We hide our polynomials at τ in these groups using the CRS.



14. The Deep Dive: Pairing Equation

The Verifier's Check:

$$e(A, B) = e(\alpha, \beta) \cdot e(y, \gamma) \cdot e(C, \delta)$$

- **LHS** ($e(A, B)$): The encrypted product of the Left and Right inputs.
- **RHS** ($e(y, \dots) \cdot e(C, \dots)$): The encrypted representation of the Output and the "DNA" of the circuit.
- $\alpha, \beta, \gamma, \delta$: These ensure the Prover didn't just pick random points.



15. The Exchange (Protocol Flow)

Prover (Witness w , CRS, Public y) $\xrightarrow{\pi=(A,B,C)}$ Verifier (Public y)

- **The Proof (π):** Exactly three Elliptic Curve points.
- $A = [A(\tau) + \alpha]_1$ (Left inputs shifted by Knowledge key).
- $B = [B(\tau) + \beta]_2$ (Right inputs shifted).
- $C = [\dots / \delta]_1$ (Outputs + Quotient $H(x)$ shifted).



16. Why is this a ZK-SNARK?

- **Zero-Knowledge:** No info about x is leaked (hidden in exponents).
- **Succinct:** ~200 bytes. Constant-time verification.
- **Non-interactive:** One single message from Prover $\rightarrow$ Verifier.
- **Argument of Knowledge:** Only someone who truly "knows" w can produce a valid (A, B, C) .



17. Strength & Weakness of Groth16

- **Strengths:**
 - **Smallest Proofs:** 3 points (Champ).
 - **Lowest Gas:** ~200k Gas to verify on Ethereum.
- **Weaknesses:**
 - **Rigidity:** One Trusted Setup ceremony per circuit change.
 - **Malleability:** A valid proof can be "tweaked" to stay valid.
 - **Quantum Safety:** Vulnerable to Shor's algorithm (Discrete Log).



18. Successor 1: PLONK (The Flexible Option)

- **Introduced:** 2019.
- **The "Universal" SNARK:**
 - Perform **one** Trusted Setup ceremony for a max size (e.g., 1B gates).
 - Use it for **any** circuit smaller than that.
- **Mechanism:** Uses permutation arguments (wire-shuffling) instead of QAP.
- **Trade-off:** Proofs are ~400–800 bytes (2-4x larger than Groth16).



19. Successor 2: STARK (The Heavyweight)

- **Introduced:** 2018.
- **No Trusted Setup:** Completely transparent (No Toxic Waste).
- **Quantum Safe:** Uses Hash functions (FRI protocol) instead of Curves.
- **Trade-off: Huge Proofs** (~45KB–100KB).
- **Analogy:** STARKs are bodybuilders—incredible power but too big to fit through small doors (Ethereum gas limits).



20. Comparison Table

Feature	Groth16	PLONK	STARK
Proof Size	~200 B (Champ)	~400-800 B	~45-100 KB
Setup	Per-Circuit (Toxic)	Universal	None
Quantum Safe	No	No	Yes
Gas Cost	Lowest	Low	High



21. The Future: SNARK on STARK

- **Hybrid Approach:**
 - i. Generate a **STARK** proof off-chain (Fast Proving, No Setup).
 - ii. Wrap that STARK in a **Groth16** SNARK (Smallest verification size).
 - iii. Verify the SNARK on-chain.
- **Logic:** You get STARK's speed and Groth16's tiny gas bill.
- **Outcome:** Best of both worlds—Scalability + Cost Efficiency.



Questions?

Thank you for your time.

Resources:

- [Circom Documentation](#)
- [RareSkills: Groth16 Math](#)
- [Vitalik Buterin: ZK-SNARKs Intro](#)



Lagrange graphs examples

- <https://www.desmos.com/calculator/ast4kyqi30>
- <https://www.desmos.com/calculator/mpntgfmvjv>
- <https://www.desmos.com/calculator/unpsbc7wdb>